

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #5: Monitoreo dinámico (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 29 de octubre
de 2024

Contenido

OBJETIVO	3
DIFICULTADES ENCONTRADAS	3
CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)	4
CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)	11
Conclusiones	12

OBJETIVO

El objetivo del proyecto es desarrollar una aplicación cliente-servidor en Java que permita a los clientes enviar periódicamente las especificaciones de hardware y datos de rendimiento de su sistema al servidor. Esto incluye información como el modelo de procesador, memoria libre, capacidad de disco, y ancho de banda disponible. El servidor recibe y almacena estos datos, permitiendo así supervisar el estado de varios sistemas conectados en una red en tiempo real.

DIFICULTADES ENCONTRADAS

1. Problemas de Conexión (ConnectException): Uno de los principales desafíos ha sido resolver el error `java.net.ConnectException: Connection timed out: connect`. Fue debido a la mala configuración de la red, firewalls, o una configuración incorrecta de la IP o el puerto en el cliente.
2. Recopilación de Datos de Hardware: Nuestro principal y único mayor problema fue la biblioteca utilizada para obtener datos de hardware (oshi). Tuvimos muchos problemas de errores de type donde no lo reconocía.

Encontramos varios retos al utilizar la biblioteca Oshi para extraer las especificaciones de hardware del sistema. Uno de los mayores desafíos fue la compatibilidad con Eclipse. Ocupábamos muchas otras librerías mas los errores que aparecían. Al final la solución fue muy sencilla, borramos el modulo de java y se solucionó.

Por último, hubo dificultades en la ejecución en diferentes entornos de desarrollo, ya que al probar OSHI en máquinas virtuales o en hardware físico, descubrimos que algunas métricas no se reportaban correctamente en entornos virtualizados. Esto complicó la simulación de escenarios de prueba y la replicación de resultados, especialmente al trabajar en diferentes sistemas. Sin embargo, pese a estos retos, logramos configurar y ajustar OSHI de manera efectiva para cumplir los objetivos del proyecto, usando intervalos adecuados de actualización y gestionando las limitaciones de compatibilidad en cada entorno.

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)

SERVIDOR

```
1 package Ej;
2
3 import oshi.SystemInfo;
4 import oshi.hardware.CentralProcessor;
5 import oshi.hardware.GlobalMemory;
6 import oshi.hardware.HardwareAbstractionLayer;
7 import oshi.hardware.HWDiskStore;
8 import oshi.hardware.NetworkIF;
9 import oshi.software.os.OperatingSystem;
10
11 import javax.swing.*;
12 import javax.swing.table.DefaultTableModel;
13 import javax.swing.table.TableRowSorter;
14 import java.awt.*;
15 import java.io.*;
16 import java.net.*;
17 import java.util.*;
18 import java.util.concurrent.*;
19 import java.util.List;
20 import java.io.File;
21
22
23
24 public class Server {
25     private static final int INTERVALO_ACTUALIZACION = 5000; //
26     private static final int PUERTO_ESTATICO = 5355; // Puerto estático
27     private static Map<String, ClienteHandler> clientesConectados = new ConcurrentHashMap<>();
28     private static DefaultTableModel tablaModelo;
29     private static JTable tabla;
30
31     public static void main(String[] args) {
32         String[] nombresColumnas = {
33             "Nombre", "Cliente/Servidor", "Procesador", "Velocidad (GHz)",
34             "Núcleos", "Disco Duro Total (GB)", "SO", "CPU Libre (%)",
35             "Memoria Libre (GB)", "Ancho Banda Libre (%)", "Disco Libre (GB)", "Estado"
36         };
37
38         --
39         tablaModelo = new DefaultTableModel(nombresColumnas, 0) {
40             @Override
41             public boolean isCellEditable(int row, int column) {
42                 return false;
43             }
44         };
45         tabla = new JTable(tablaModelo);
46
47         // Configurar el frame
48         JFrame ventana = new JFrame("Servidor");
49         ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
50         ventana.add(new JScrollPane(tabla));
51         ventana.setSize(1200, 400);
52         ventana.setVisible(true);
53
54         // Configurar el servidor
55         try (ServerSocket serverSocket = new ServerSocket(PUERTO_ESTATICO)) { // Usar puerto estático
56             System.out.println("Servidor iniciado en el puerto: " + PUERTO_ESTATICO);
57
58             // Agregar datos del servidor
59             agregarDatosDelServidor();
60
61             // Hilo para actualizar datos del servidor y ordenar filas
62             ScheduledExecutorService programador = Executors.newScheduledThreadPool(1);
63             programador.scheduleAtFixedRate(() -> {
64                 actualizarDatosDelServidor();
65                 actualizarDatosDeClientes();
66                 ordenarTablaPorCpuLibre();
67             }, 0, INTERVALO_ACTUALIZACION, TimeUnit.MILLISECONDS);
68
69             // Aceptar conexiones de clientes
70             while (true) {
71                 Socket socketCliente = serverSocket.accept();
72                 ClienteHandler handler = new ClienteHandler(socketCliente);
```

```

72         new Thread(handler).start();
73     }
74     } catch (Exception e) {
75         e.printStackTrace();
76     }
77 }
78
79 private static void agregarDatosDelServidor() {
80     SystemInfo sistemaInfo = new SystemInfo();
81     HardwareAbstractionLayer hal = sistemaInfo.getHardware();
82     CentralProcessor procesador = hal.getProcessor();
83     OperatingSystem sistemaOperativo = sistemaInfo.getOperatingSystem();
84     List<HWDiskStore> discos = hal.getDiskStores();
85
86     String modeloProcesador = procesador.getProcessorIdentifier().getName();
87     double velocidadProcesador = procesador.getProcessorIdentifier().getVendorFreq() / 1e9;
88     int numeroNucleos = procesador.getLogicalProcessorCount();
89     long tamanoDisco = discos.get(0).getSize() / (1024 * 1024 * 1024);
90
91     tablaModelo.addRow(new Object[]{
92         "SERVIDOR",
93         "Servidor",
94         modeloProcesador,
95         velocidadProcesador,
96         numeroNucleos,
97         tamanoDisco,
98         sistemaOperativo.toString(),
99         0.0, // CPU Libre
100         0.0, // Memoria Libre
101         0.0, // Ancho de banda
102         0.0, // Disco libre
103         "CONECTADO"
104     });
105 }
106
107 private static void actualizarDatosDelServidor() {
108     SystemInfo sistemaInfo = new SystemInfo();
109     HardwareAbstractionLayer hal = sistemaInfo.getHardware();
110     CentralProcessor procesador = hal.getProcessor();
111     GlobalMemory memoria = hal.getMemory();
112     List<HWDiskStore> discos = hal.getDiskStores();
113
114     // Calcular CPU libre
115     long[] ticksPrevios = procesador.getSystemCpuLoadTicks();
116     try { Thread.sleep(1000); } catch (InterruptedException e) { }
117     double cpuLibre = 100.0 - (procesador.getSystemCpuLoadBetweenTicks(ticksPrevios) * 100);
118
119     // Calcular memoria libre
120     double memoriaLibre = memoria.getAvailable() / (1024.0 * 1024 * 1024);
121
122     // Calcular ancho de banda libre (aproximado basado en la utilización de red)
123     final double anchoBandaLibre;
124     List<NetworkIF> redes = hal.getNetworkIFs();
125     if (!redes.isEmpty()) {
126         NetworkIF red = redes.get(0);
127         red.updateAttributes();
128         long bytesTotales = red.getBytesRecv() + red.getBytesSent();
129         anchoBandaLibre = 100.0 - (bytesTotales % 100); // Simplificado para ejemplo
130     } else {
131         anchoBandaLibre = 100.0;
132     }
133
134     // Calcular espacio libre en disco
135     File discoRaiz = new File("/");
136     double discolibre = discoRaiz.getUsableSpace() / (1024.0 * 1024 * 1024); // Convertimos a GB
137
138     // Actualizar fila del servidor
139     SwingUtilities.invokeLater(() -> {
140         tablaModelo.setValueAt(String.format("%.2f", cpuLibre), 0, 7);
141         tablaModelo.setValueAt(String.format("%.2f", memoriaLibre), 0, 8);
142         tablaModelo.setValueAt(String.format("%.2f", anchoBandaLibre), 0, 9);
143         tablaModelo.setValueAt(String.format("%.2f", discolibre), 0, 10);
144     });
145 }

```

```

146
147 private static void actualizarDatosDeClientes() {
148     // Crear una lista temporal de clientes a remover
149     List<String> clientesAEliminar = new ArrayList<>();
150
151     // Verificar clientes desconectados
152     clientesConectados.forEach((id, handler) -> {
153         if (!handler.isConnected()) {
154             actualizarEstadoCliente(id, "DESCONECTADO");
155             clientesAEliminar.add(id);
156         }
157     });
158
159     // Remover clientes desconectados
160     clientesAEliminar.forEach(clientesConectados::remove);
161
162     // Actualizar la tabla solo para clientes conectados
163     for (int i = 0; i < tablaModelo.getRowCount(); i++) {
164         String idCliente = (String) tablaModelo.getValueAt(i, 0);
165         String estado = (String) tablaModelo.getValueAt(i, 11);
166
167         // Si es el servidor, continuar con el siguiente
168         if (idCliente.equals("SERVIDOR")) {
169             continue;
170         }
171
172         // Si el cliente está desconectado, mantener los valores pero en gris
173         if (estado.equals("DESCONECTADO")) {
174             tabla.setRowSelectionInterval(i, i);
175             tabla.setSelectionBackground(Color.LIGHT_GRAY);
176         }
177     }
178 }
179
180 private static void actualizarEstadoCliente(String idCliente, String estado) {
181     for (int i = 0; i < tablaModelo.getRowCount(); i++) {
182         if (tablaModelo.getValueAt(i, 0).equals(idCliente)) {
183             tablaModelo.setValueAt(estado, i, 11);
184
185             if (estado.equals("DESCONECTADO")) {
186                 // Limpiar todos los campos excepto ID y tipo de cliente
187                 tablaModelo.setValueAt("", i, 2); // Modelo Procesador
188                 tablaModelo.setValueAt(0.0, i, 3); // Velocidad
189                 tablaModelo.setValueAt(0, i, 4); // Núcleos
190                 tablaModelo.setValueAt(0.0, i, 5); // Disco Duro Total
191                 tablaModelo.setValueAt("", i, 6); // SO
192                 tablaModelo.setValueAt(0.0, i, 7); // CPU Libre
193                 tablaModelo.setValueAt(0.0, i, 8); // Memoria Libre
194                 tablaModelo.setValueAt(0.0, i, 9); // Ancho Banda Libre
195                 tablaModelo.setValueAt(0.0, i, 10); // Disco Libre
196
197                 // Cambiar color de fondo a gris
198                 tabla.setRowSelectionInterval(i, i);
199                 tabla.setSelectionBackground(Color.LIGHT_GRAY);
200             } else {
201                 tabla.clearSelection();
202                 tabla.setSelectionBackground(tabla.getBackground());
203             }
204             break;
205         }
206     }
207     ordenarTablaPorCpuLibre();
208 }
209
210 private static void ordenarTablaPorCpuLibre() {
211     TableRowSorter<DefaultTableModel> ordenado = new TableRowSorter<>(tablaModelo);
212
213     // Configurar comparador personalizado
214     ordenado.setComparator(7, Comparator.comparingDouble(value -> {
215         if (value instanceof String) {
216             return Double.parseDouble((String) value);
217         }

```

```

218         return 0.0;
219     }).reversed());
220
221     tabla.setRowSorter(ordenado);
222 }
223
224 private static class ClienteHandler implements Runnable {
225     private Socket socket;
226     private boolean conectado;
227
228     public ClienteHandler(Socket socket) {
229         this.socket = socket;
230         this.conectado = true;
231         String idCliente = "CLIENTE" + clientesConectados.size();
232         clientesConectados.put(idCliente, this);
233     }
234
235     @Override
236     public void run() {
237         try (BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
238             PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {
239             out.println("Conectado como " + socket.getInetAddress() + ":" + socket.getPort());
240
241             String mensaje;
242             while ((mensaje = in.readLine()) != null) {
243                 System.out.println("Mensaje recibido de " + socket.getInetAddress() + ": " + mensaje);
244                 // Aquí puedes implementar la lógica de manejo del mensaje del cliente
245             }
246         } catch (IOException e) {
247             System.err.println("Error en la conexión del cliente: " + e.getMessage());
248         } finally {
249             conectado = false;
250             try {
251                 socket.close();
252             } catch (IOException e) {
253                 e.printStackTrace();
254             }
255         }
256     }
257
258     public boolean isConnected() {
259         return conectado;
260     }
261 }
262 }
263

```

CLIENTE

```
1 package Ej;
2
3 import oshi.SystemInfo;
4 import oshi.hardware.*;
5 import oshi.software.os.OperatingSystem;
6
7 import javax.swing.*;
8 import javax.swing.table.DefaultTableModel;
9 import java.io.*;
10 import java.net.Socket;
11 import java.util.HashMap;
12 import java.util.List;
13 import java.util.Map;
14
15 public class Client {
16     private static final int UPDATE_INTERVAL = 5000; // 5 segundos
17     private static final String SERVER_IP = "25.47.153.25"; // Cambia esto a la IP del servidor
18     private static final int SERVER_PORT = 5355; // Cambia esto al puerto del servidor
19
20     public static void main(String[] args) throws InterruptedException {
21         try {
22             String clientId = getHostName();
23
24             // Conectar al servidor
25             try (Socket socket = new Socket(SERVER_IP, SERVER_PORT);
26                 ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream())) {
27
28                 // Crear y iniciar el hilo de actualización
29                 Thread updateThread = createUpdateThread(oos, clientId);
30                 updateThread.start();
31
32                 System.out.println("Sigues conectado al servidor");
33                 System.out.println("\nPresiona ENTER para desconectarte");
34                 System.in.read(); // Esperar entrada del usuario
35
36                 // Interrumpir el hilo y esperar su finalización
37                 updateThread.interrupt();
38                 try {
39                     updateThread.join(); // Esperar a que el hilo termine
40                 } catch (InterruptedException e) {
41                     System.out.println("El hilo fue interrumpido mientras esperaba su finalización.");
42                     Thread.currentThread().interrupt(); // Mantener estado de interrupción
43                 }
44
45                 } catch (IOException e) {
46                     System.err.println("Error al conectar o enviar datos al servidor: " + e.getMessage());
47                 }
48
49             } catch (IOException e) {
50                 System.err.println("Error al obtener el nombre del host: " + e.getMessage());
51             }
52         }
53
54     private static String getHostName() throws IOException {
55         return java.net.InetAddress.getLocalHost().getHostName();
56     }
57
58     private static Thread createUpdateThread(ObjectOutputStream oos, String clientId) throws InterruptedException {
59         return new Thread(() -> {
60             try {
61                 while (!Thread.currentThread().isInterrupted()) {
62                     Map<String, Object> specs = getSystemSpecs(clientId);
63                     oos.writeObject(specs);
64                     oos.reset(); // Limpiar cache del stream
65
66                     showSpecsInTable(specs); // Mostrar especificaciones en una tabla
67                     sleepSafely(UPDATE_INTERVAL); // Esperar el intervalo de actualización
68                 }
69             }
70         });
71     }
72 }
```



```

69         } catch (IOException e) {
70             System.out.println("Error al enviar datos al servidor: " + e.getMessage());
71         }
72     });
73 }
74
75 private static Map<String, Object> getSystemSpecs(String clientId) {
76     SystemInfo si = new SystemInfo();
77     HardwareAbstractionLayer hal = si.getHardware();
78     CentralProcessor processor = hal.getProcessor();
79     GlobalMemory memory = hal.getMemory();
80     List<HWDiskStore> diskStores = hal.getDiskStores();
81     List<NetworkIF> networks = hal.getNetworkIFs();
82     OperatingSystem os = si.getOperatingSystem();
83
84     long[] prevTicks = processor.getSystemCpuLoadTicks();
85     sleepSafely(1000); // Esperar un segundo
86
87     double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
88     double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
89     double bandwidthFree = calculateBandwidthFree(networks);
90     double diskFree = calculateDiskFree(diskStores);
91
92     Map<String, Object> specs = new HashMap<>();
93     specs.put("ID", clientId);
94     specs.put("Modelo del Procesador", processor.getProcessorIdentifier().getName());
95     specs.put("Velocidad del Procesador (GHz)", processor.getProcessorIdentifier().getVendorFreq() / 1e9);
96     specs.put("Núcleos", processor.getLogicalProcessorCount());
97     specs.put("Capacidad del Disco Duro (GB)", diskStores.isEmpty() ? 0 : diskStores.get(0).getSize() / (1024
98     specs.put("Versión del Sistema Operativo", os.toString());
99     specs.put("CPU Libre", String.format("%.2f", cpuFree));
00     specs.put("Memoria Libre", String.format("%.2f", freeMemory));
01     specs.put("Ancho de Banda Libre", String.format("%.2f", bandwidthFree));
02     specs.put("Disco Libre", String.format("%.2f", diskFree));
103
104     return specs;
105 }
106
107 private static double calculateBandwidthFree(List<NetworkIF> networks) {
108     if (!networks.isEmpty()) {
109         NetworkIF net = networks.get(0);
110         net.updateAttributes(); // Actualizar atributos de la red
111         long totalBytes = net.getBytesRecv() + net.getBytesSent();
112         return 100.0 - (totalBytes % 100); // Calcular el ancho de banda libre
113     }
114     return 100.0; // Si no hay redes disponibles
115 }
116
117 private static double calculateDiskFree(List<HWDiskStore> diskStores) {
118     if (!diskStores.isEmpty()) {
119         HWDiskStore diskStore = diskStores.get(0);
120         long totalDiskSpace = diskStore.getSize();
121         long usedDiskSpace = diskStore.getReadBytes() + diskStore.getWriteBytes();
122         return (totalDiskSpace - usedDiskSpace) / (1024.0 * 1024 * 1024); // Calcular espacio libre en disco
123     }
124     return 0; // No hay discos disponibles
125 }
126
127 private static void sleepSafely(long milliseconds) {
128     try {
129         Thread.sleep(milliseconds); // Dormir el hilo
130     } catch (InterruptedException e) {
131         Thread.currentThread().interrupt(); // Mantener estado de interrupción
132     }
133 }
134
135 private static void showSpecsInTable(Map<String, Object> specs) {
136     DefaultTableModel model = new DefaultTableModel();
137     model.addColumn("Característica");
138     model.addColumn("Valor");
139 }

```

```

140     for (Map.Entry<String, Object> entry : specs.entrySet()) {
141         model.addRow(new Object[]{entry.getKey(), entry.getValue()}); // Agregar fila por cada especificación
142     }
143
144     JTable table = new JTable(model);
145     table.setFillsViewportHeight(true);
146     JScrollPane scrollPane = new JScrollPane(table);
147
148     JPanel panel = new JPanel();
149     panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
150     panel.add(scrollPane);
151
152     JDialog dialog = new JDialog();
153     dialog.setTitle("Especificaciones del Sistema");
154     dialog.setModal(true);
155     dialog.setContentPane(panel);
156     dialog.pack();
157     dialog.setLocationRelativeTo(null);
158     dialog.setVisible(true); // Mostrar el diálogo
159 }
160 }

```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)

SERVIDOR

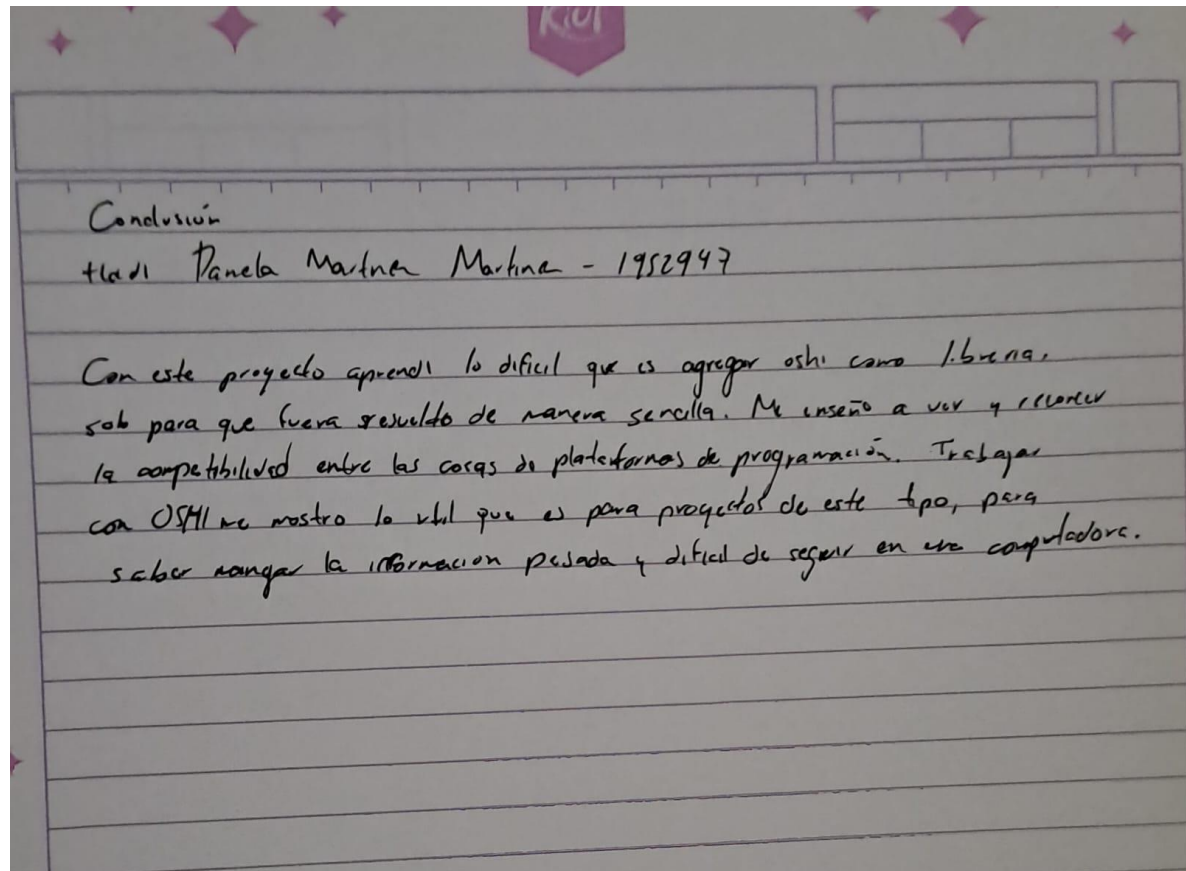
Servidor										
Nombre	Cliente/Servidor	Procesador	Velocidad (GHz)	Núcleos	Disco Duro Total..	SO	CPU Libre (%)	Memoria Libre (...)	Ancho Banda Lib...	Disco Libre (GB)
RVIDOR	Servidor	11th Gen Intel(R)	2.688	12	476	Microsoft Windo...	97.35	2.47	100.00	72.93

CLIENTE

Especificaciones del Sistema	
Característica	Valor
Disco Libre	420.95
Núcleos	12
Velocidad del Procesador (GHz)	2.688
CPU Libre	94.10
Capacidad del Disco Duro (GB)	476
Ancho de Banda Libre	100.00
Modelo del Procesador	11th Gen Intel(R) Core(TM) i5-11400H @ ..
ID	HM
Memoria Libre	2.05
Versión del Sistema Operativo	Microsoft Windows 11 (Home) build 22631

Conclusiones

1952947 - Heidi Pamela Martínez Martínez



Andrea Monserrat Méndez Flores
Conclusión - 1845517

Para esta actividad fue necesario obtener los datos del cliente, los cuales fueron acomodados en una tabla, esta tabla fue necesario investigar como se pudiera ir actualizando automáticamente con información en tiempo real, cambiando constantemente.

Tarea practica #5 Monitoreo Dinámico

Alexia Hernandez Andrade 2109147

En conclusión la segunda parte de lo que es el monitoreo dinámico nos permitió ampliar conocimientos ya que implementar la actualización periódica de los datos como el porcentaje de CPU libre, la memoria disponible, el ancho de banda y el espacio del disco duro nos acerca a la comprensión de como realizar un monitoreo en tiempo real, lo cual es importante para la materia. Además integrar el estado de conexión fue algo complicado que permitió entender mejor los aspectos de conectividad.

Alejandro Sauqué Pérez 2004428 ITS
Conclusion Individual Sistemas Distribuidos y Paralelos

En esta práctica tuvimos que buscar una manera de hacer lo mismo que la práctica pasada pero dinámicamente. Considero que este comportamiento que ocurre en la comunicación de cliente-servidor es demasiado útil porque la mayoría de los sistemas necesitan de la lectura o escritura frecuente de los datos. Podríamos fácilmente imaginar que esta práctica nos puede llevar al intercambio de información en tiempo real. Vemos este tipo de tecnología en muchas apps hoy en día. Y a pesar de que esta práctica no la hace en tiempo real sí nos hace ver la noción de cómo funciona un sistema así.

Conclusión

El objetivo de la práctica o actividad era que los datos de la máquina fueran registrados y tuvieran algunos cambios en tiempo real si era el caso. Se tuvo la complicación de que no aceptaba la librería oshy - oshi, realmente si nos tomó bastante tiempo el descubrir el error pero al final si estuvo todo bien con la librería, fue una práctica complicada pero se logró entender el funcionamiento y como resolver los problemas de conexión.

Se buscaba que se tuviera la actualización de los datos recibidos por el servidor en la tabla dinámica, con respecto a tiempo, logrando los resultados requeridos.